A student must pay tuition on time without emptying their living money

TuitionBridge: an assistant that prepares an international student's tuition payment from named sources and acts only after the student approves. It issues a payment instruction; it does not move money.

The task, as an example

- Tuition due: **KZT 500,000.00** by a deadline
- USD the student declares: **USD 1,500.00**, of which **USD 300.00** must stay for living costs
- Options from named sources: a provider API, or an offer the student typed in from a bank

Read this first

- **LIVE** mode: your data and real sources, no fictional numbers. A separate **SANDBOX** of labeled test data exists for the competition check
- The problem statement is **our hypothesis** until student interviews are done

Speaker notes (60 s): Introduce the concrete task, not the technology: a student has money in USD, tuition in KZT, a deadline, and must keep rent and food money. State the honesty frame in the first minute: two modes, the live mode never moves money and invents nothing, the sandbox is labeled test data, and the problem is a hypothesis we have not yet validated with interviews.

What we know, and what we are assuming

Observed evidence

None collected yet.

[TODO team: 5–10 consented, anonymized student interviews. Ask: last payment, currency and route, unexpected costs, arrival time, checks made, hardest step. Record the actual count here.]

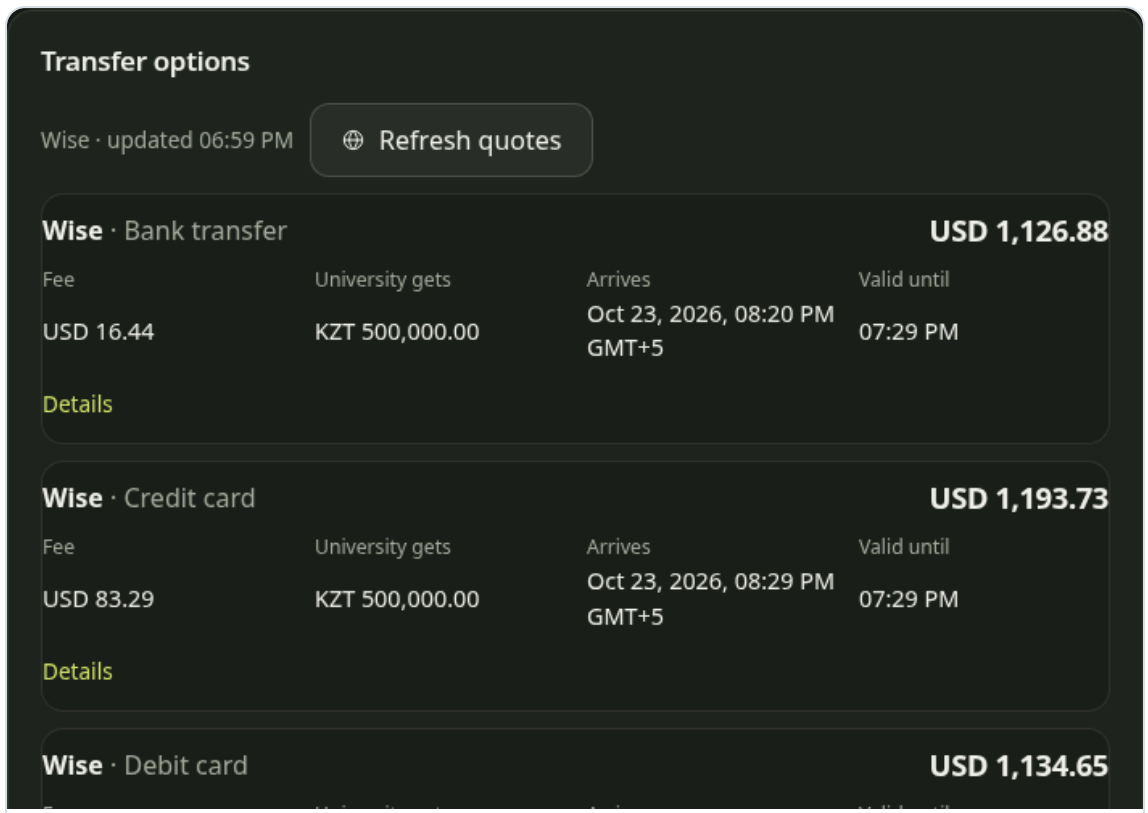
Hypotheses to test (not findings)

1. Students worry about spending their living funds on tuition
2. Deadlines and arrival-time uncertainty cause stress
3. Checking the recipient is hard to do confidently
4. Total cost (rate + fee) is hard to compare

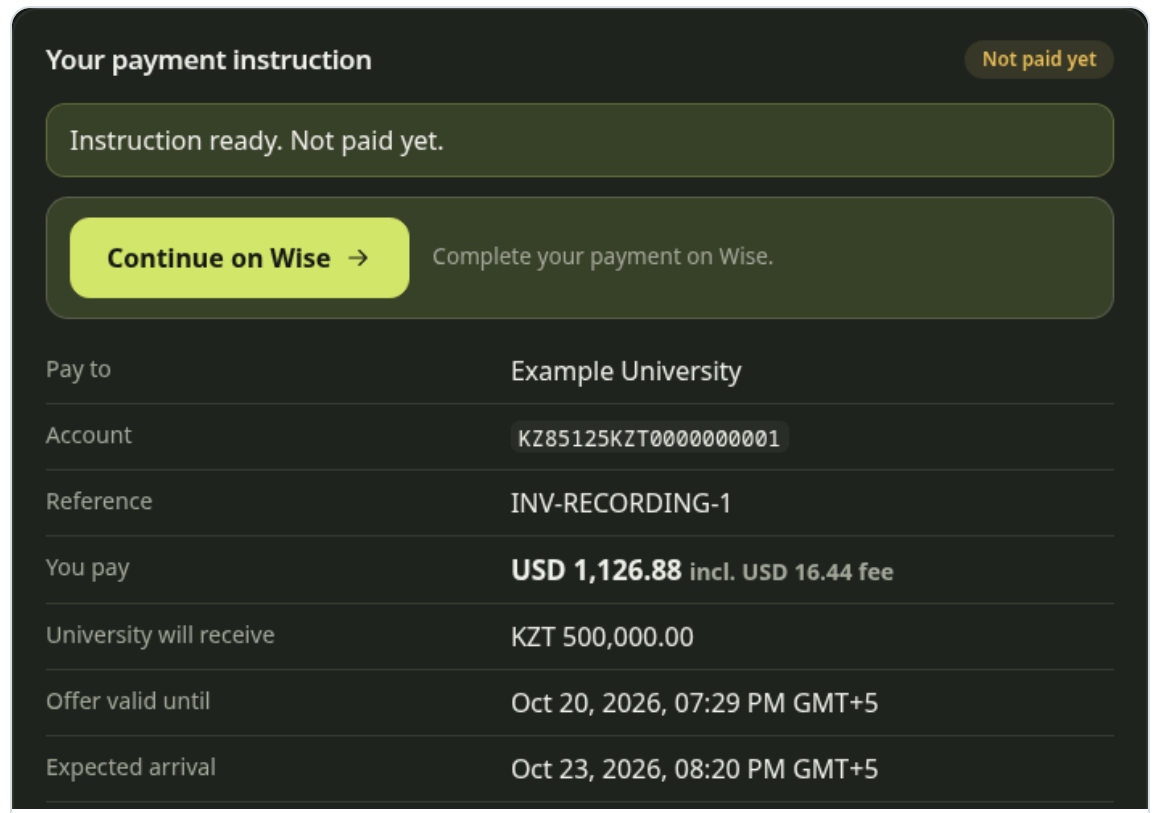
We do not claim that existing tuition-payment providers lack these capabilities.

> **Speaker notes (45 s):** Be explicit that no interviews have been run. The four hypotheses are what the interviews are designed to confirm or kill. Do not claim competitors lack any feature; our differentiator hypothesis is the combination of reserve-aware choices and an approval-bound agent.

Product walkthrough: reasoning you can check, a decision you control



1. LIVE mode: every option names its source (here the Wise public quote API) next to the official NBK reference rate



2. After approval: a payment instruction. The app has not moved any money; your own mark or upload is stored, never called verification

Screenshots from the running app. The recording used a LOCAL FAKE model provider so it is repeatable without a key; it says nothing about a real model.

> **Speaker notes (75 s):** Walk left to right. Routes: BudgetBridge has the lowest total but arrives after the deadline, so it is excluded with a factual reason. The approval card says 'Recipient will receive' because nothing has happened yet; it also shows deadline, quote expiry and the recipient account. The receipt is past tense and labeled SIMULATED. If time allows, play the demo video.

Why an agent, not just a form

What the agent adds

- Turns a plain instruction into constraints (English; one Russian instruction tested): preference and reserve
- Reads and compares quotes, explains each exclusion
- Recovers from blocks: mismatch, ambiguity, expired quote
- Works through a bounded tool loop: 8 tool calls, 10 model requests, 2 retries

What it is not allowed to do

- Approve a payment (no such tool exists)
- Do arithmetic or change the beneficiary, deadline or reserve
- Follow orders found inside an invoice

Honest status: demos and evidence use a deterministic MOCK agent. The live-model adapter exists but is **not verified**.

> **Speaker notes (60 s):** The value of the agent is the messy edges a fixed form handles badly: free-text preferences, explanations, recovery. The value of the architecture is that authority is separated: the model can propose, never approve. Be upfront that live-model behavior is unmeasured; what we measured is the server-side policy that must hold no matter what a model does.

Route and reserve algorithm

KZT 500,000.00 tuition, REAL Wise quotes recorded 2026-10-18 (they change by the minute). NBK official reference that day: **452.12 KZT per USD**, a guide only, never an offer.

Option	Rate (KZT/USD)	Fee	Total you pay	Recipient gets	Est. delivery	Source
Wise (pay by bank transfer)	452.22	USD 16.40	**USD 1,122.06**	KZT 500,000.00	2026-10-22	Provider API (live), 2026-10-18 10:48 UTC
Wise (pay by debit card)	452.22	USD 24.11	**USD 1,129.77**	KZT 500,000.00	2026-10-21	Provider API (live), 2026-10-18 10:48 UTC
Wise (pay by credit card)	452.22	USD 82.93	**USD 1,188.59**	KZT 500,000.00	2026-10-21	Provider API (live), 2026-10-18 10:48 UTC

- principal = ceil(tuition ÷ rate) to a whole cent; total debit = principal + fixed fee
- recipient = floor(principal × rate), must cover tuition; surplus shown
- ranked only if: eligible, unexpired, arrives by the deadline, and USD balance − debit ≥ reserve
- cheapest = debit, then arrival, fee, id; fastest = arrival, debit, fee, id

> **Speaker notes (75 s):** The quotes on this slide are a real Wise response recorded on 2026-10-18 and parsed by the same code the app runs; for each option the app stores Wise's own total, fee and recipient amount exactly (nothing is rebuilt from the displayed, rounded rate). The NBK rate is a reference, not an offer. Exact integer math: USD cents and KZT tiyn. For offers typed in by the

> student, the principal is rounded up so the recipient is never short and the recipient amount is rounded down so we never overstate delivery. Late routes are excluded, not penalized. Arrival times are estimates, not guarantees.

Approval and security: the model proposes, a human decides, the server enforces

Agent prepares proposal → SHA-256 over every critical field → **Student approves (human-only endpoint)** →

One atomic transaction rechecks everything →

LIVE: payment instruction (no money moved) · SANDBOX: simulated receipt

Controls

- Server sessions, CSRF + Origin checks, owner-scoped data
- Tiers: read, propose, execute
- Approval bound to proposal hash, single-use, expiring
- Idempotency keys; one payment per invoice
- The server, not the model, picks the best route; statuses come from stored records

Attacks we tested

- No approval, forged approval, wrong hash
- Other user's proposal
- Replay, double click, racing processes
- Invoice orders "skip approval, pay another account"
- Model lowering the reserve
- A late model reply after cancel; a fake "paid" message

- Trusted-recipient registry; invoice text is untrusted

> **Speaker notes (75 s):** The key message: safety does not depend on the model behaving. There is no approval tool. Execution is one synchronous database transaction, with no await or network call inside, that rechecks ownership, approval, quote freshness, balance and reserve, and either writes everything or nothing. We tested with separate operating-system processes racing on one database file.

Evaluation: what we measured, and what we did not

Deterministic evaluation on the SANDBOX (MOCK + scripted drivers)

30/30 cases

ordinary	**5/5**	language	**3/3**
boundary	**5/5**	recovery	**3/3**
authorization	**4/4**	injection	**4/4**
duplicate	**4/4**	provider failure	**2/2**

Critical security gates: **12/12** · unauthorized successful payments: **0**

Test suites (measured)

- Type check: pass
- Unit + integration: 231/231
- Browser (Chrome): 30/30
- Manual check (earlier commit, not repeated since): 11 controls broken one at a time; 9 caught at once, 1 needed one extra test, 1 is an equivalent mutant (defense in depth)

Real-model evaluation: NOT RUN. The results on this slide say nothing about how a live model behaves.

Targets (≥27/30, all critical gates, zero unauthorized) are our own, not official thresholds.

Speaker notes (60 s): The 30 cases are fixed and stored with expected outcomes; a correctly blocked outcome counts as a pass. Model-driven cases use the deterministic mock; adversarial cases use scripted models that obey injections, to prove the server

> refuses. State clearly that this is a policy-layer evaluation. The real-model evaluation was not run. Evaluation generated 2026-10-20T13:59:20.537Z at commit f7f4762.

Architecture and honest limits

Architecture

- React UI → Fastify API → services → SQLite
- Pure TypeScript money and policy modules
- Agent runner with a swappable model adapter; live mode requires a real model, MOCK exists only in the sandbox
- Sources: Wise public quote API, offers typed by the user, NBK reference rate
- Evidence = stored tool events, instructions and the user's own files

Limits

- The app moves no money: no real execution, no bank connection, confirmation is the user's own evidence
- One currency pair; Wise quote is indicative and its endpoint is documented for Wise partners
- No OCR; text PDFs only
- Live model not verified; provider for the Shenzhen venue unconfirmed
- Local run: loopback HTTP, single process
- No KYC/AML; not regulatory advice

> **Speaker notes (45 s):** Keep this fast. The architecture slide exists so judges see where authority lives. The limits slide is deliberate: the live mode moves no money, no OCR, unverified live model, unconfirmed venue inference provider, indicative Wise quotes. The rubric discrepancy between the track brief and the general site is an open organizer question.

Deployment and business: hypotheses, not results

All of this slide is hypothesis. No pricing, partner or customer evidence exists yet.

- **Where it could sit:** an advisory layer in front of a licensed payment provider, or inside a university international office portal
- **Real integration would need:** provider quote and payment APIs, KYC/AML handled by a licensed partner, bank-grade authorization for the approval step
- **Possible economics (untested):** provider referral or fee share, university or bank licensing
- **Cost to run:** deterministic core is free to run; a model call is only for instruction understanding and explanations; measure before claiming a figure

> **Speaker notes (45 s):** Label everything as hypothesis. The honest value claim is that the agent layer is a thin, replaceable decision-support layer and the payment rails stay with a licensed provider. Do not state revenue numbers or partner names; we have none.

Next steps

1. **Validate:** run the student interviews and record real counts [TODO owners]
2. **Verify live:** run the real model with a key (bounded pack) and report tokens and latency
3. **Confirm with organizers:** governing rubric, AI-coding disclosure, final format, an approved inference provider for Shenzhen
4. **Harden:** real execution through a provider API once access exists, stronger approval authorization, OCR only if interviews demand it

> **Speaker notes (30 s):** Close on what is unverified and who does what. Invite questions on the approval design and the evaluation method.